BPMS - Process Mining II

Prof. Dr. Andreas Hense
andreas.hense@h-brs.de

Wirtschaftsinformatik

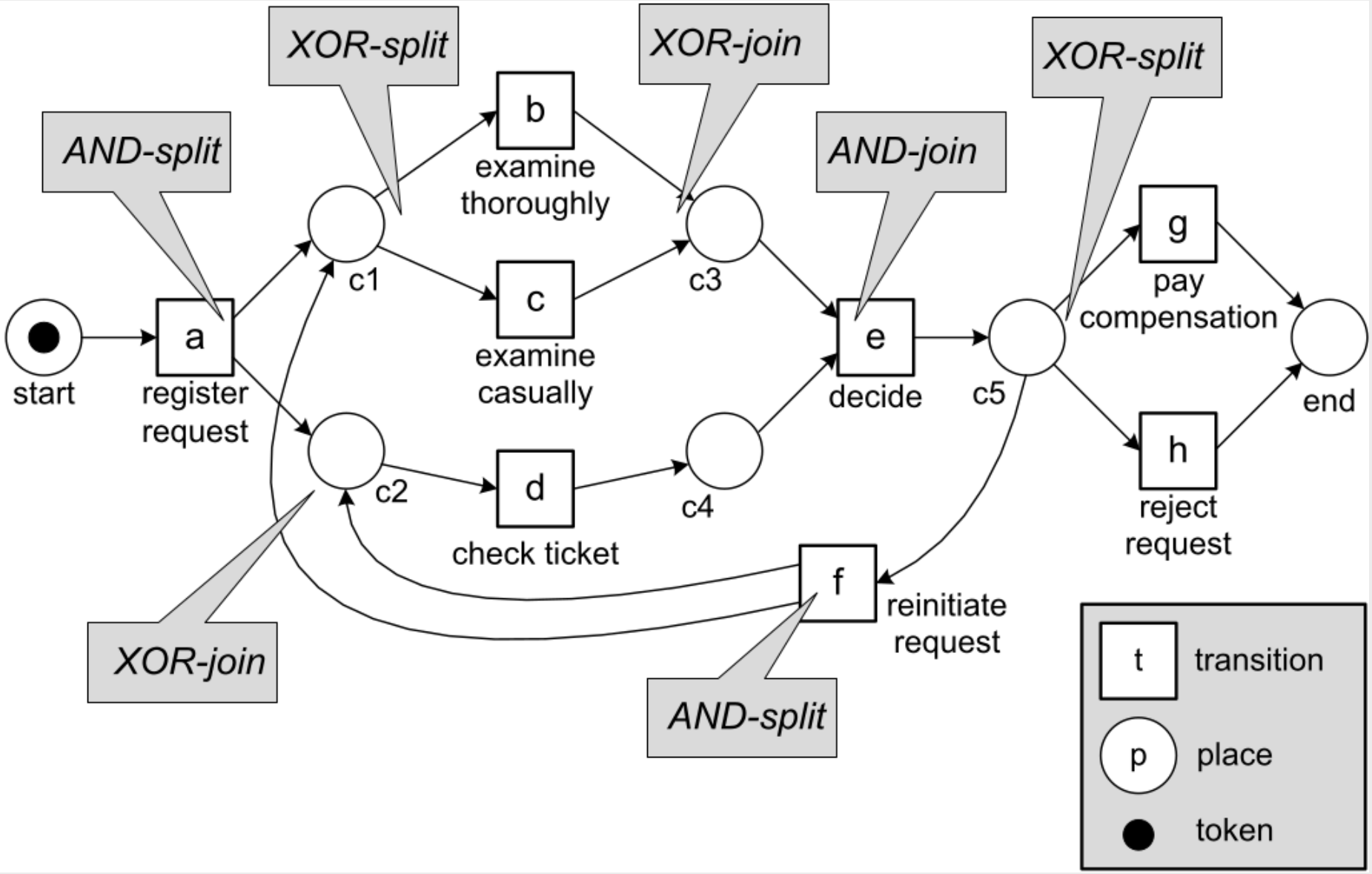
Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

Agenda I

1. Causal Nets
2. Process Mining Algorithmen
   - Heuristic Miner
   - Inductive Miner
   - Fuzzy Miner
3. Conformance Checking
4. Vorgehensweise beim Process Mining
5. Literatur

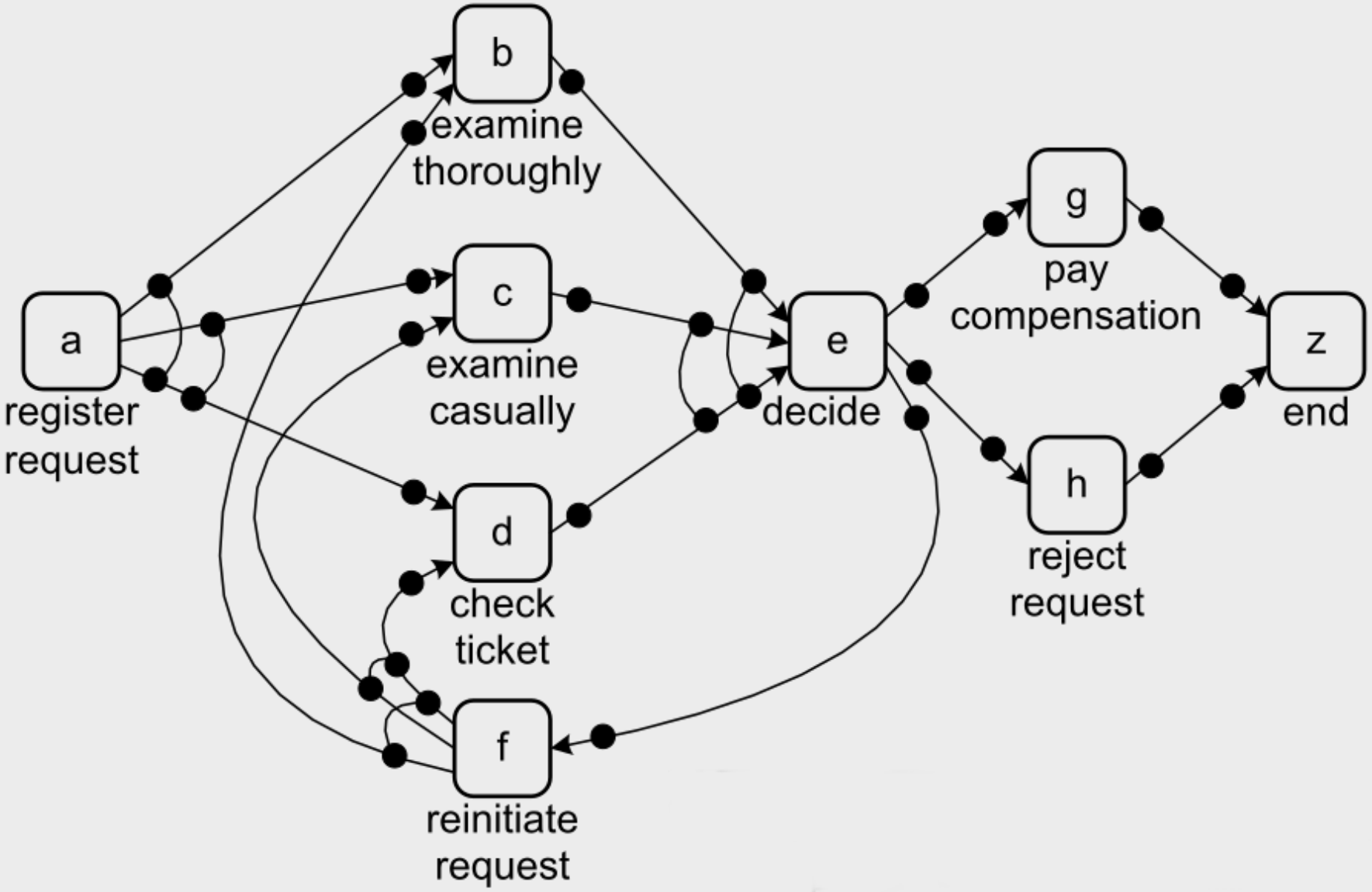
- In den Petrinetz-basierten Prozessmodellierungssprachen wie EPK, BPMN und YAWL werden Aktivitäten mit Stellen und Operatoren verbunden.
- Während Aktivitäten Spuren im Log erzeugen, gibt es über Stellen und Operatoren keine direkten Informationen.
- Es gibt daher Mining-Algorithmen, wie den Fuzzy- oder den Heuristic-Miner, die nur Aktivitäten und keine verbindenden Elemente nutzen.
- Causal Nets sind speziell für Process-Mining konstruiert worden.
- Bei einem Causal Net sind die Knoten Aktivitäten, und die Kanten repräsentieren kausale Abhängigkeiten.
- Wenn man "z" als eine künstlich hinzugefügte, stille Transition begreift, haben das Petrinetz und das Causal Net danach die gleiche Menge von Traces.

Petrinetz



[Aal16]

Causal Net: Beispiel



[Aal16]

Definition Causal Net I

Definition

Ein Causal Net ist ein Tupel $C = (A, a_i, a_o, D, I, O)$ wobei:

$A \subseteq \mathcal{A}$ endliche Menge von Aktivitäten,

$a_i \in A$ Start Aktivität,

$a_o \in A$ End Aktivität,

$D \subseteq A \times A$ Abhängigkeitsrelation,

$AS = \{X \subseteq \mathcal{P}(A) \mid X = \{\emptyset\} \vee \emptyset \notin X\}$ Teilmenge der Potenzmenge,

$I \in A \rightarrow AS$ Menge der möglichen Input-Bindings pro Aktivität,

$O \in A \rightarrow AS$ Menge der möglichen Output-Bindings pro Aktivität.

so dass

- $D = \{(a_1, a_2) \in A \times A \mid a_1 \in \bigcup_{as \in I(a_2)} as\}$,
- $D = \{(a_1, a_2) \in A \times A \mid a_2 \in \bigcup_{as \in O(a_1)} as\}$,

Definition Causal Net II

- $\{a_i\} = \{a \in A \mid I(a) = \{\emptyset\}\}$,
- $\{a_o\} = \{a \in A \mid O(a) = \{\emptyset\}\}$, und
- alle Aktivitäten im Graphen (A, D) befinden sich auf einem Pfad von a_i nach a_o.

Causal Net: Bindung I

Wir definieren eine Aktivitätsbindung, um zu beschreiben, dass eine Aktivität mit einem bestimmten Input- und Output-Binding stattgefunden hat.

Definition

Sei $C = (A, a_i, a_o, D, I, O)$ ein Causal Net.

$$B = \{(a, as^I, as^O) \in A \times \mathcal{P}(A) \times \mathcal{P}(A) \mid as^I \in I(a) \wedge as^O \in O(a)\}$$

ist die Menge derAktivitäts-Bindungen. Eine Bindungs-Sequenz σ ist eine Sequenz von Aktivitäts-Bindungen, d. h. $\sigma \in B^*$.

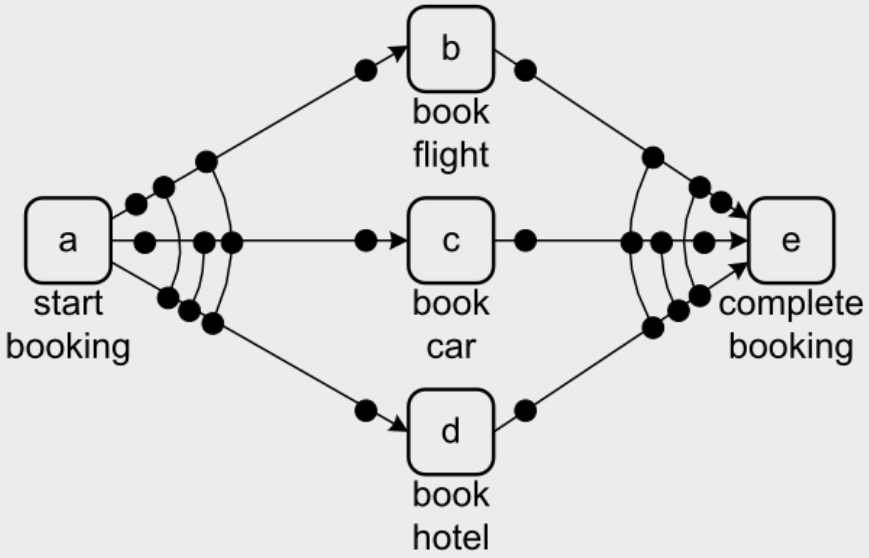
- Bindungs-Sequenzen sind valide, wenn die Vorgänger- und Nachfolger-Aktivitäten in den Bindungen "übereinstimmen".
- Um das zu formalisieren, wird der Zustandsraum eines Causal Net als Multimenge aller Verpflichtungen definiert.
- Wir schreiben Multimengen (Bag) mit $\mathbb{B}()$.

Definition

- Sei $C = (A, a_i, a_o, D, I, O)$ ein Causal Net.
- $S = \mathbb{B}(A \times A)$ ist der Zustandsraum von C.
- $s \in S$ ist ein Zustand, d. h. eine Multimenge von Verpflichtungen.
- Die Funktion $\Psi \in B^* \to S$ wird induktiv definiert:
- $\Psi(\langle\rangle) = []$ und
- $\Psi(\sigma \oplus (a, as^I, as^O)) = (\Psi(\sigma) \backslash (as^I \times \{a\})) \uplus (\{a\} \times as^O)$ für jede Bindungs-Sequenz $\sigma \oplus (a, as^I, as^O) \in B^*$.
- $\Psi(\sigma)$ ist der Zustand nach Ausführen der Bindungs-Sequenz σ.

- Eine valide Bindungs-Sequenz beginnt mit der Start-Aktivität a_i, endet mit der Ende-Aktivität a_O, entfernt nur Verpflichtungen, die offen sind, und endet ohne offene Verpflichtungen.
- Folgendes ist ein Beispiel für eine valide Bindungs-Sequenz mit den Aktivitäten abde:

Bindungs-Sequenz und Verpflichtungen



$$\psi(\langle\,\rangle) = [\,]$$

$$\psi(\langle(a, \emptyset, \{b, d\})\rangle) = [(a, b), (a, d)]$$

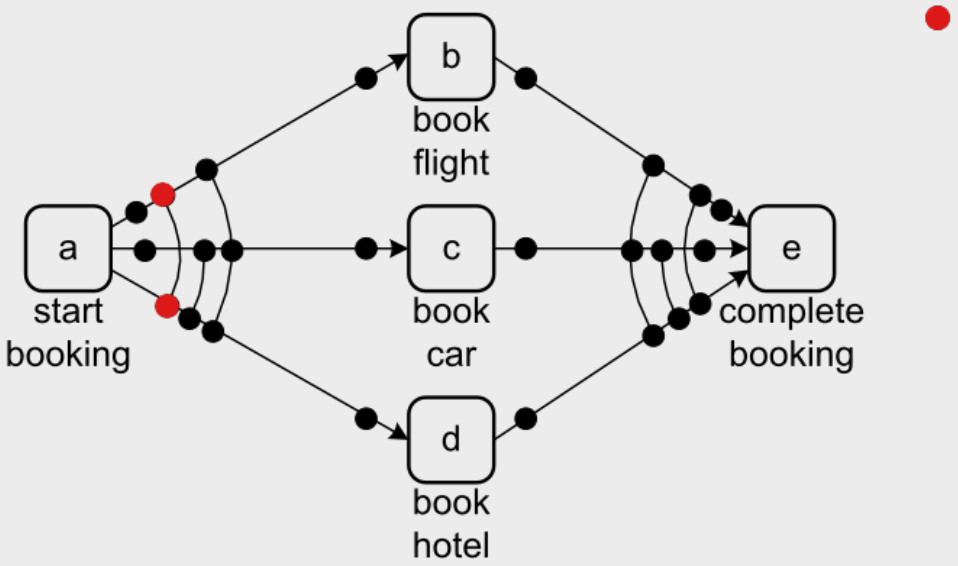
$$\psi(\langle(a, \emptyset, \{b, d\}), (d, \{a\}, \{e\})\rangle) = [(a, b), (d, e)]$$

$$\psi(\langle(a, \emptyset, \{b, d\}), (d, \{a\}, \{e\}), (b, \{a\}, \{e\})\rangle) = [(b, e), (d, e)]$$

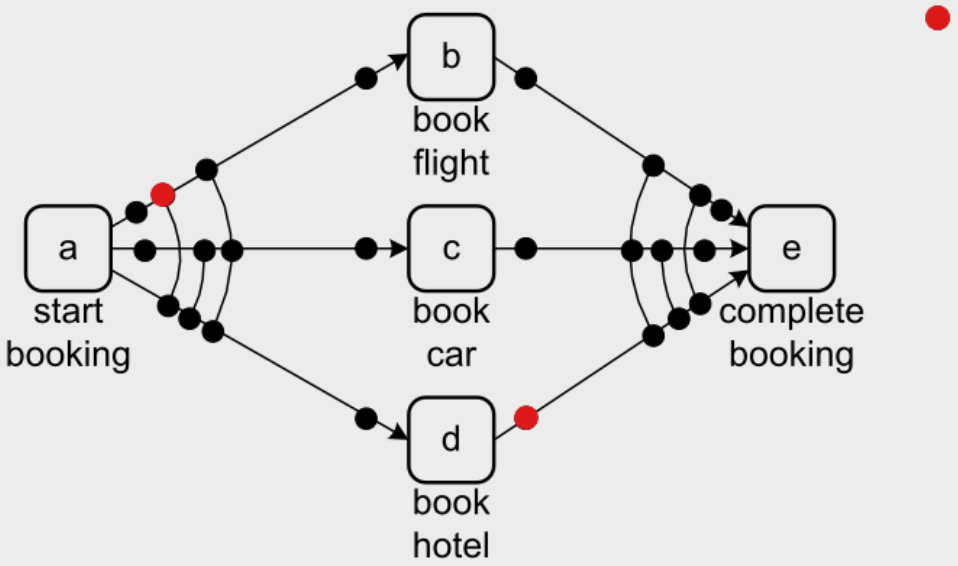
$$\psi(\langle(a, \emptyset, \{b, d\}), (d, \{a\}, \{e\}), (b, \{a\}, \{e\}), (e, \{b, d\}, \emptyset)\rangle) = [\,]$$

[Aal16]

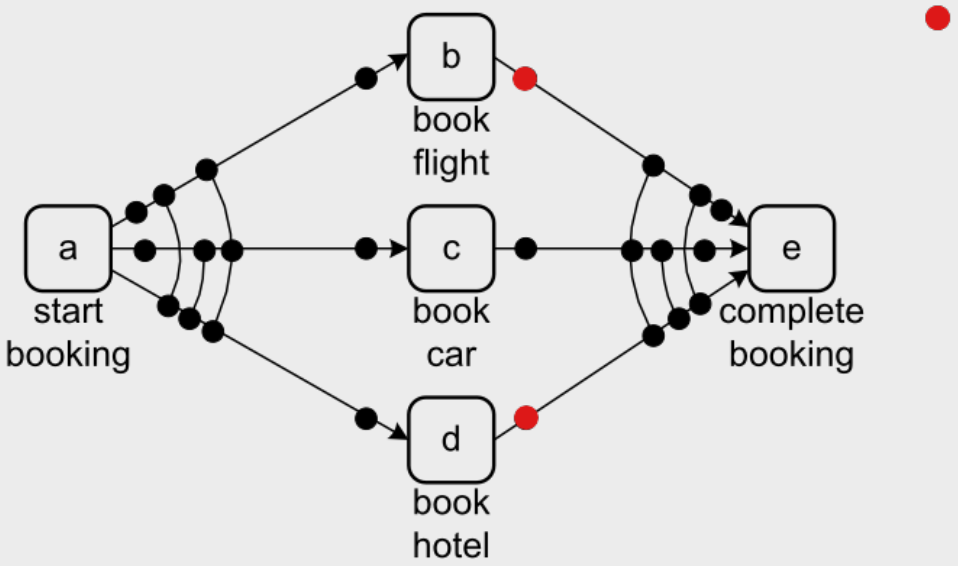
adbe


b
book
flight
a
start
booking
c
book
car
e
complete
booking
d
book
hotel

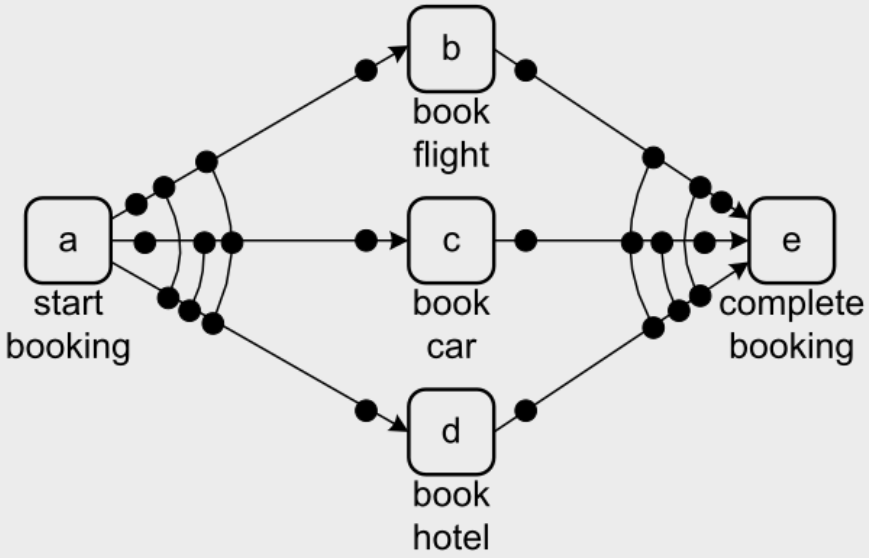
adbe


b
book
flight
a
start
booking
c
book
car
e
complete
booking
d
book
hotel

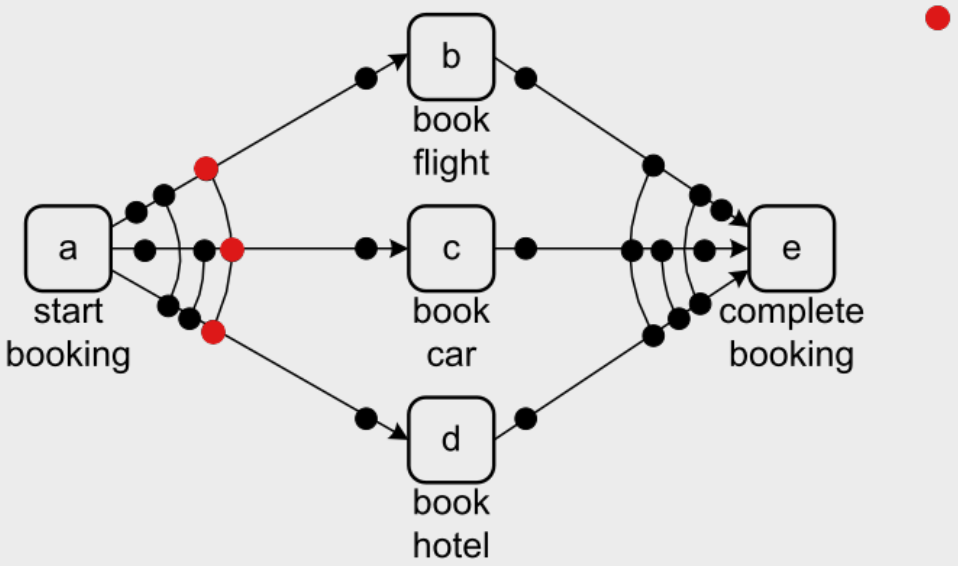
adbe


b
book
flight
a
start
booking
c
book
car
e
complete
booking
d
book
hotel

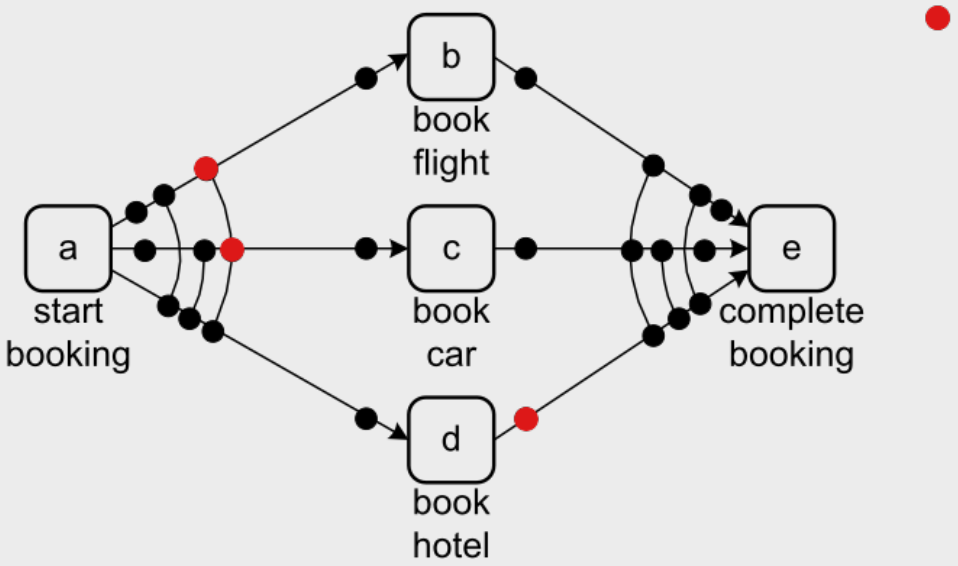
adbe


a
start
booking
b
book
flight
c
book
car
d
book
hotel
e
complete
booking

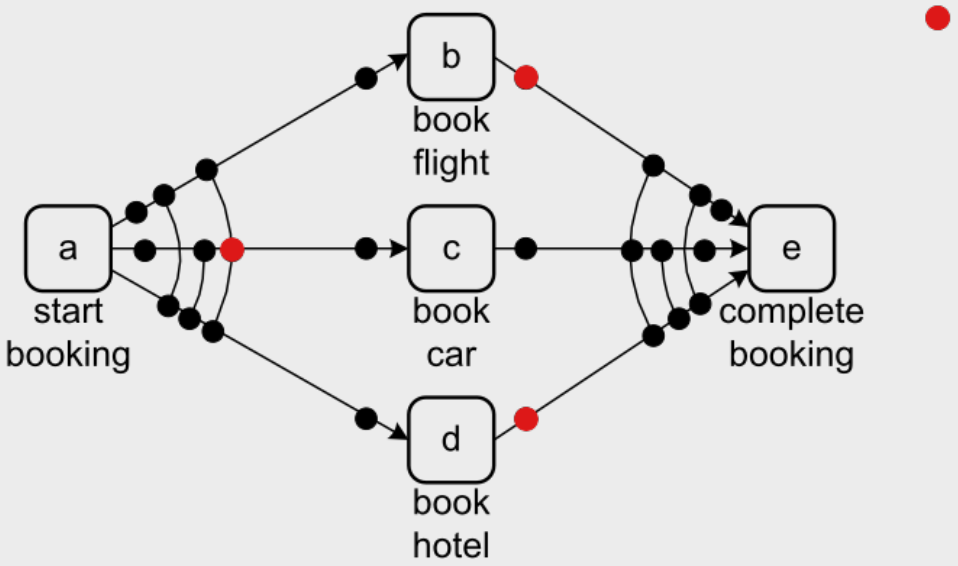
adbce


a
start
booking
b
book
flight
c
book
car
d
book
hotel
e
complete
booking

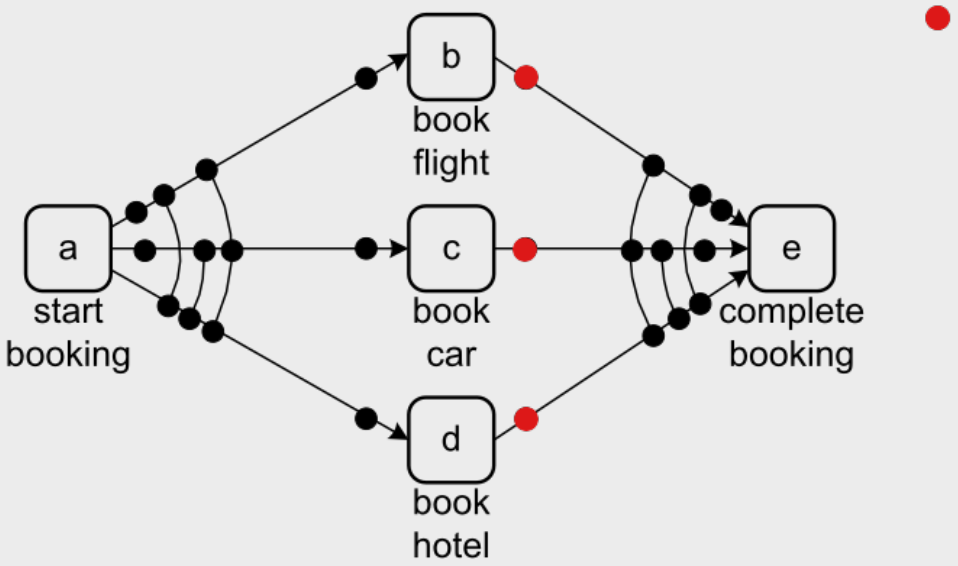
adbce


a
start
booking
b
book
flight
c
book
car
d
book
hotel
e
complete
booking

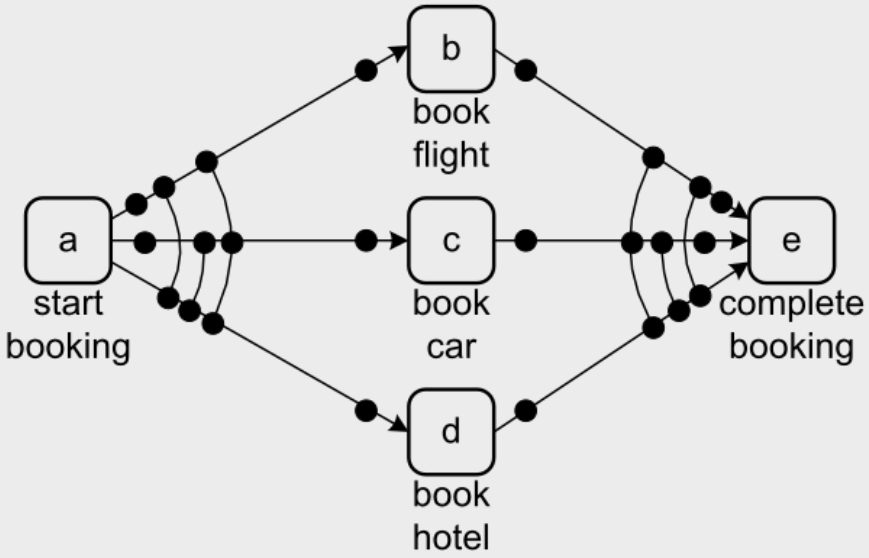
adbce


b
book
flight
a
start
booking
c
book
car
e
complete
booking
d
book
hotel

adbce


a
start
booking
b
book
flight
c
book
car
d
book
hotel
e
complete
booking

adbce


a
start
booking
b
book
flight
c
book
car
d
book
hotel
e
complete
booking

Definition

Sei $C = (A, a_i, a_o, D, I, O)$ ein Causal Net und $\sigma = \langle (a_1, as_1^I, as_1^O), (a_2, as_2^I, as_2^O), \ldots, (a_n, as_n^I, as_n^O) \rangle \in B^*$ eine Bindungs-Sequenz. σ ist valide gdw

- $a_1 = a_i, a_n = a_o$ und $a_k \in A \setminus \{a_i, a_o\}$ für $1 < k < n$;
- $\Psi(\sigma) = []$; und
- für jedes Präfix $\langle (a_1, as_1^I, as_1^O), (a_2, as_2^I, as_2^O), \ldots, (a_k, as_k^I, as_k^O) \rangle = \sigma' \oplus (a_k, as_k^I, as_k^O) \in pref(\sigma) : (as_k^I \times \{a_k\}) \leq \Psi(\sigma')$.

Die letzte Bedingung besagt, dass die aus $\Psi(\sigma')$ zu entfernenden Verpflichtungen $as_k^I \times \{a_k\}$ auch wirklich vorhanden sein müssen ($\leq$ auf Multimenge).

- Die Semantik von Causal Nets ist über komplette Sequenzen definiert.
- Es ist keine lokale Replay-Semantik wie bei Petrinetzen.
- Man muss bei Causal Nets also schon vorher wissen, welche Bindung die richtige für den betrachteten Trace ist.

Process Mining Algorithmen

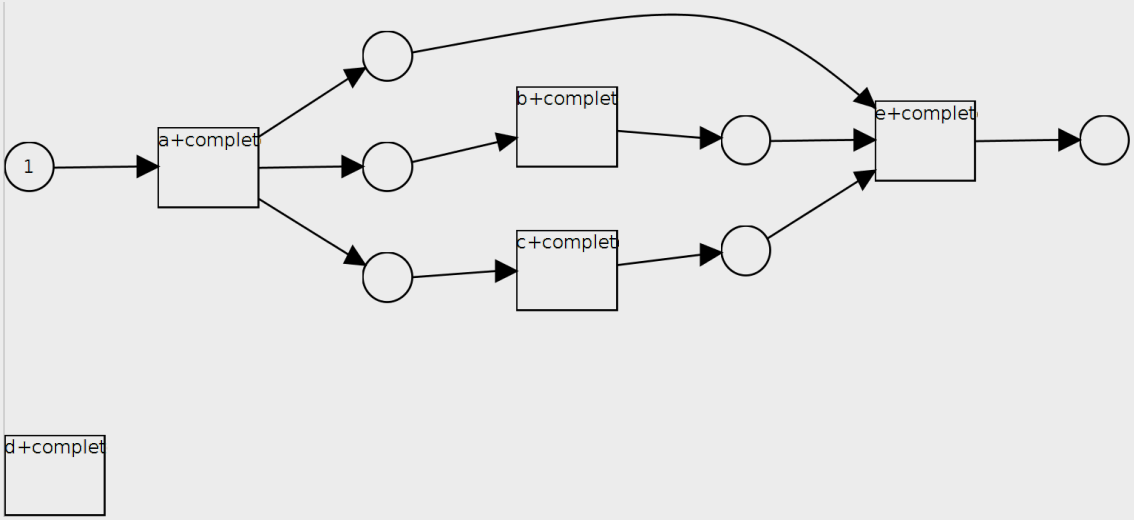
Heuristic Miner

Gegeben folgender Event Log (`.../event-logs-process-mining-book/Chapter_6/L-heur-1.xes`):

$$L = [\langle a, e\rangle^5, \langle a, b, c, e\rangle^{10}, \langle a, c, b, e\rangle^{10}, \langle a, b, e\rangle^1, \langle a, c, e\rangle^1,$$

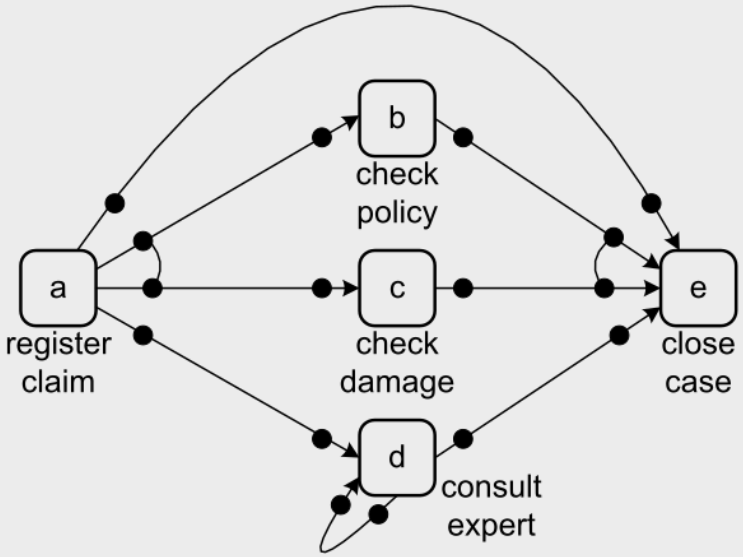
$$\langle a, d, e\rangle^{10}, \langle a, d, d, e\rangle^2, \langle a, d, d, d, e\rangle^1]$$

Der Alpha-Miner erzeugt folgendes:



Causal Net für Log L

Wie kann man folgendes Netz erhalten, wenn wir annehmen, dass alle Traces, die nur einmal vorkommen, Rauschen sind?



[Aal16]

Frequenztabelle Log L

$\lvert >_L \rvert$	a	b	c	d	e
a	0	11	11	13	5
b	0	0	10	0	11
c	0	10	0	0	11
d	0	0	0	4	13
e	0	0	0	0	0

[Aal16]

- Tabelle lässt sich mit dem ProM Plugin "Interactive Data-aware Heuristic Miner" auch als Directly Follows Graph ausgeben.

Definition

Dependency Measure Sei L ein Event Log über $\mathcal{A}$ und $a, b \in \mathcal{A}$. $|a >_L b|$ ist die Anzahl der Male, dass auf a direkt b folgt in L. Wenn σ ein Trace in L ist, dann ist $L(\sigma)$ die Anzahl der Vorkommen von σ in L.

$$|a >_L b| = \sum_{\sigma \in L} L(\sigma) \times |\{1 \leq i < |\sigma| \mid \sigma(i) = a \wedge \sigma(i+1) = b\}|$$

$|a \Rightarrow_L b|$ ist der Wert der Dependency Relation zwischen a und b:

$$|a \Rightarrow_L b| = \begin{cases} \frac{|a>_L b| - |b>_L a|}{|a>_L b| + |b>_L a| + 1}, \text{if } a \neq b \\ \frac{|a>_L a|}{|a>_L a| + 1}, \text{if } a = b \end{cases}$$

Dependency Measure für L

$\lvert\Rightarrow_L\rvert$	a	b	c	d	e
a	$\frac{0}{0+1}=0$	$\frac{11-0}{11+0+1}=0.92$	$\frac{11-0}{11+0+1}=0.92$	$\frac{13-0}{13+0+1}=0.93$	$\frac{5-0}{5+0+1}=0.83$
b	$\frac{0-11}{0+11+1}=-0.92$	$\frac{0}{0+1}=0$	$\frac{10-10}{10+10+1}=0$	$\frac{0-0}{0+0+1}=0$	$\frac{11-0}{11+0+1}=0.92$
c	$\frac{0-11}{0+11+1}=-0.92$	$\frac{10-10}{10+10+1}=0$	$\frac{0}{0+1}=0$	$\frac{0-0}{0+0+1}=0$	$\frac{11-0}{11+0+1}=0.92$
d	$\frac{0-13}{0+13+1}=-0.93$	$\frac{0-0}{0+0+1}=0$	$\frac{0-0}{0+0+1}=0$	$\frac{4}{4+1}=0.80$	$\frac{13-0}{13+0+1}=0.93$
e	$\frac{0-5}{0+5+1}=-0.83$	$\frac{0-11}{0+11+1}=-0.92$	$\frac{0-11}{0+11+1}=-0.92$	$\frac{0-13}{0+13+1}=-0.93$	$\frac{0}{0+1}=0$

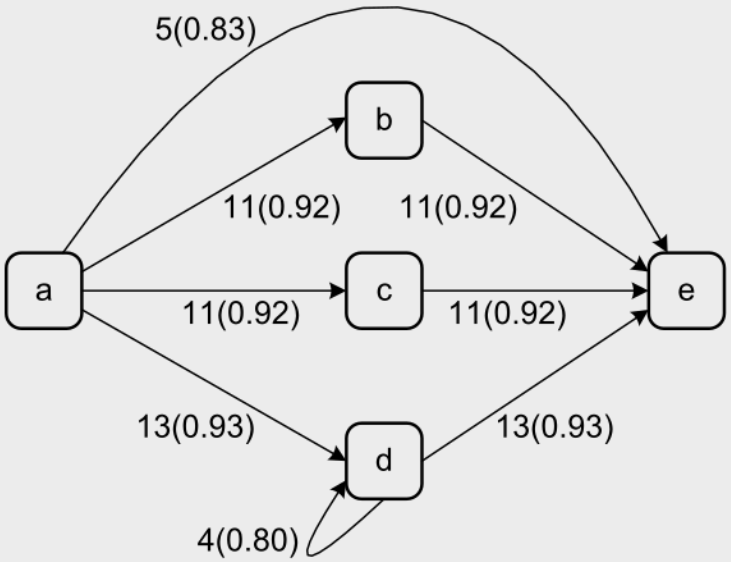
[Aal16]

- Tabelle lässt sich mit dem ProM Plugin “Interactive Data-aware Heuristic Miner” auch als Dependency Graph ausgeben.

Dependency Graph für L

Dabei wird nur eine Kante gezogen, wenn

- Der Eintrag in $| >_L |$ größer oder gleich 2
- Der Eintrag in $| \Rightarrow_L |$ größer oder gleich 0.7

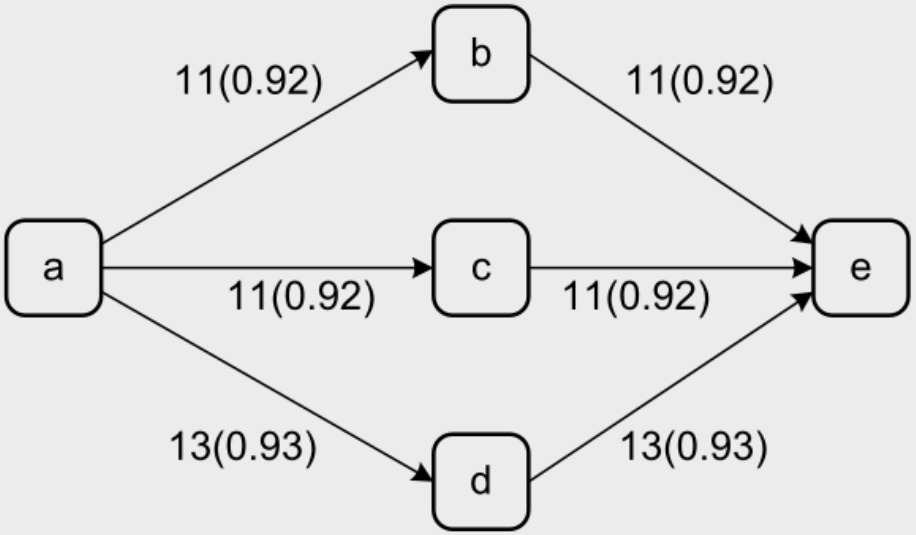


[Aal16]

Dependency Graph für L mit höherem Threshold

Dabei wird nur eine Kante gezogen, wenn

- Der Eintrag in $| >_L |$ größer oder gleich 5
- Der Eintrag in $| \Rightarrow_L |$ größer oder gleich 0.9

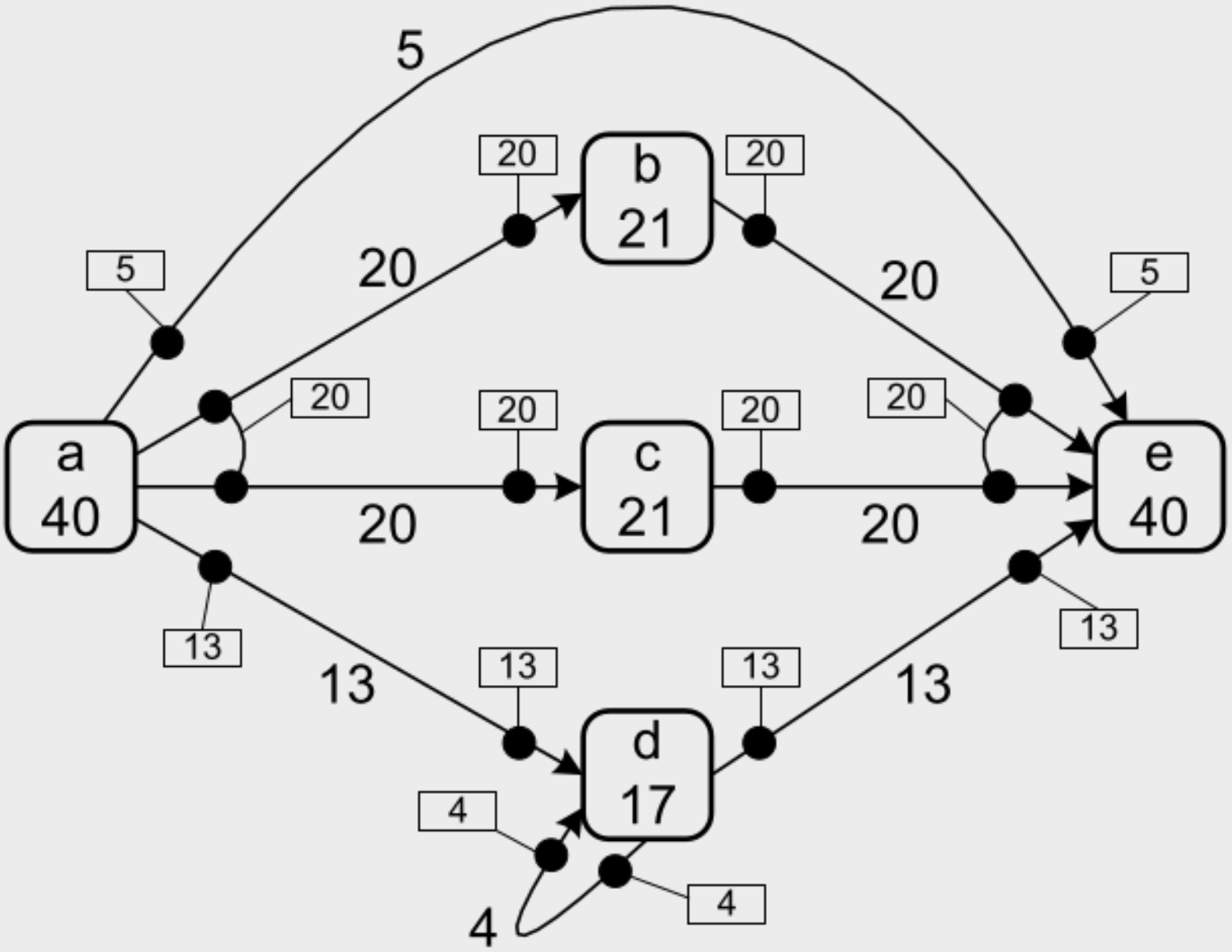


[Aal16]

- Weniger häufiges Verhalten kann vom Benutzer so ausgeblendet werden.
- Der Fuzzy-Miner (s. u.) erlaubt außerdem das "Verbergen" von selten vorkommenden Aktivitäten.

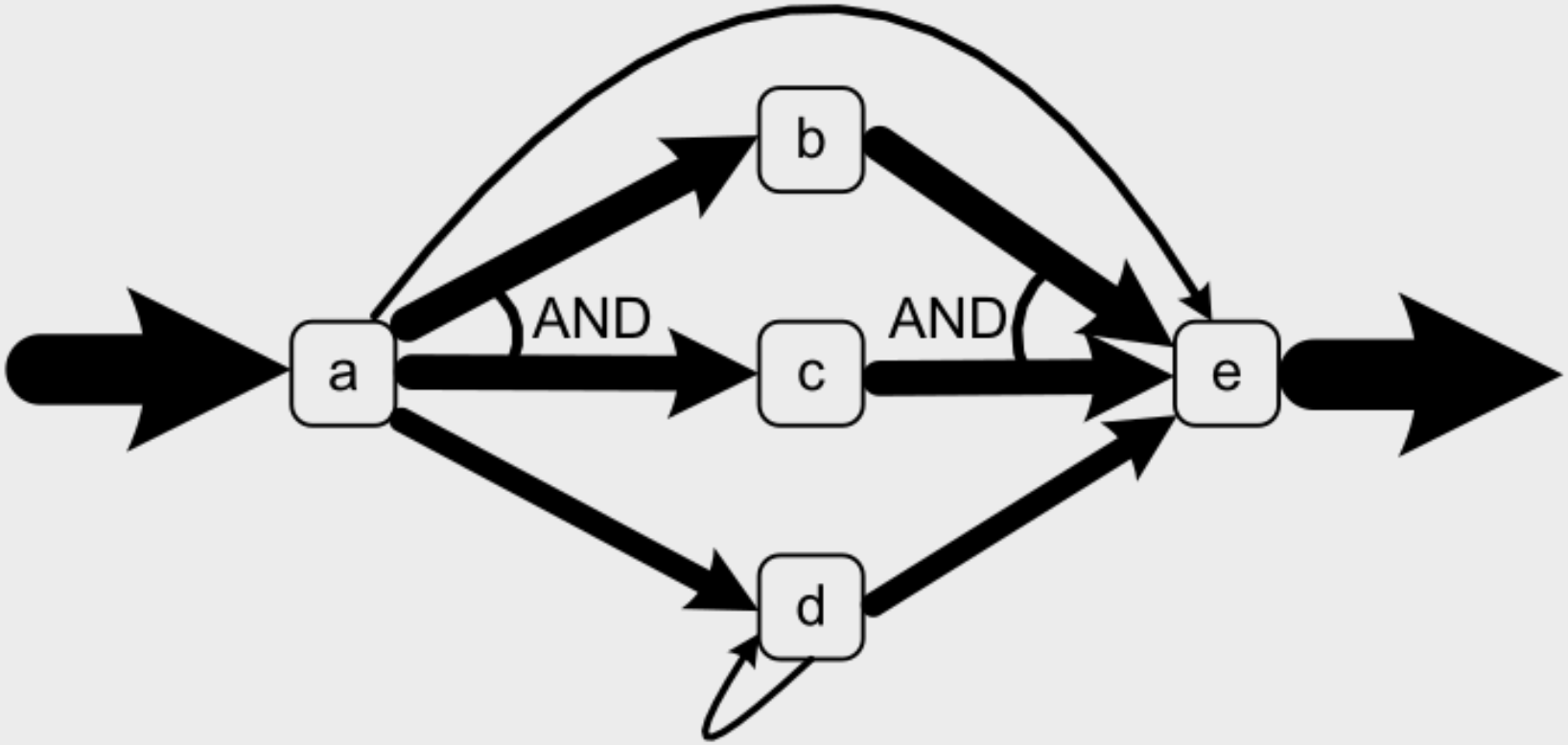
- Bisher haben wir die Aktivitäten A und die Abhängigkeitsrelation D eines Causal Nets abgeleitet.
- Wenn keine eindeutige Start- bzw. Endaktivität (a_i, a_o) vorhanden ist, muss diese künstlich angehängt werden.
- Wir müssen nun noch die Input-Bindings I und die Output-Bindings O erkennen.
- Hier kann man sich auf die Events in einem Zeitfenster vor und nach dem zu betrachtenden Event beschränken oder andere Heuristiken wählen.
- Ein möglicher Ansatz wird hier beschrieben [WR11].
- Ein Causal Net kann in ein trace-äquivalentes Petrinetz transformiert werden [AAD11].

Causal Net für L mit Werten



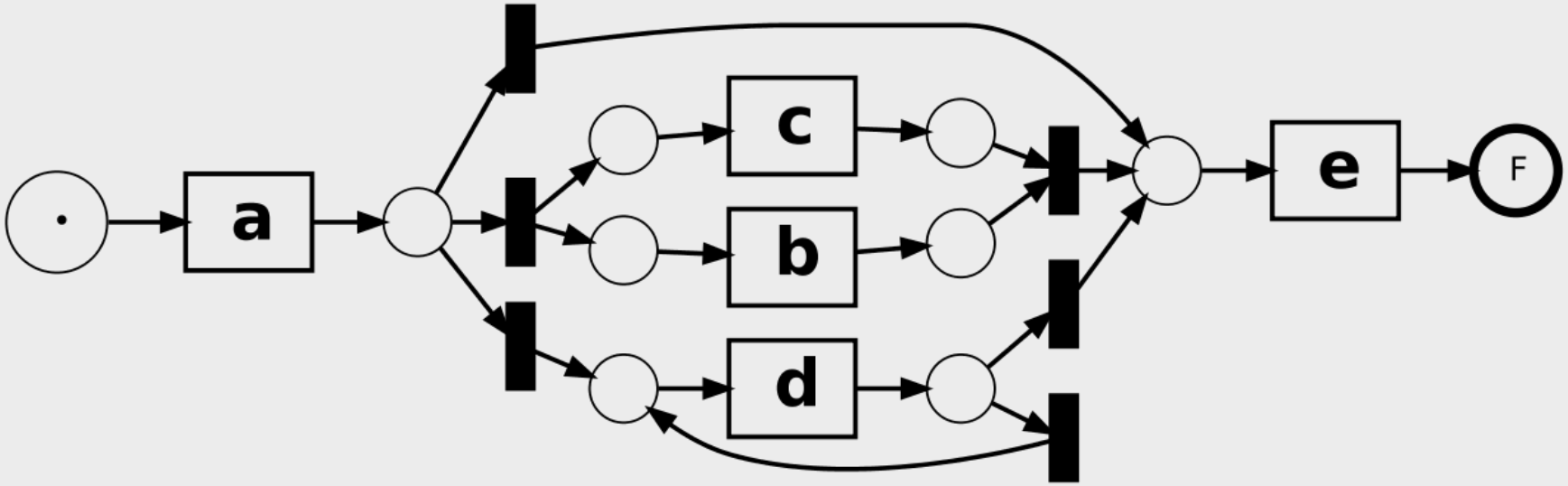
[Aal16]

Causal net für L mit gewichteten Kanten



[A

Trace-äquivalentes Petri-Netz



- Erzeugt mit dem Plugin "Interactive Data-aware Heuristic Miner".

Process Mining Algorithmen

Inductive Miner

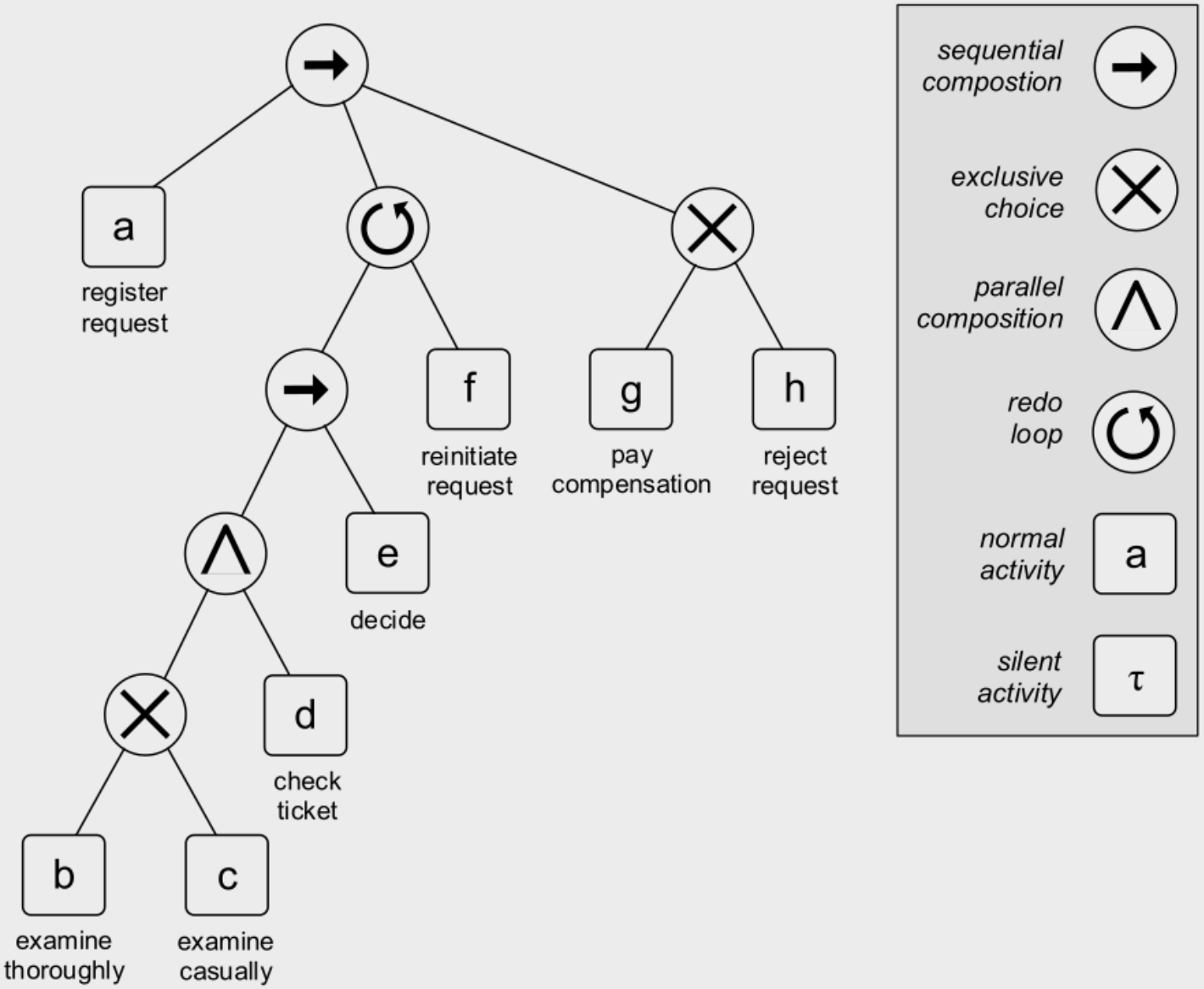
- Schneidet den Log an geeigneten Stellen auseinander und erstellt Sublogs.
- Das Ergebnis sind Process Trees, welche sich in Petrinetze verwandeln lassen, welche per Konstruktion die Soundness-Eigenschaft besitzen.

Definition

Sei $A \subseteq \mathcal{A}$ eine endliche Menge von Aktivitäten mit $\tau \notin A$.
$\bigoplus = \{\rightarrow, \times, \wedge, \circlearrowleft\}$ ist die Menge der Process Tree Operatoren.

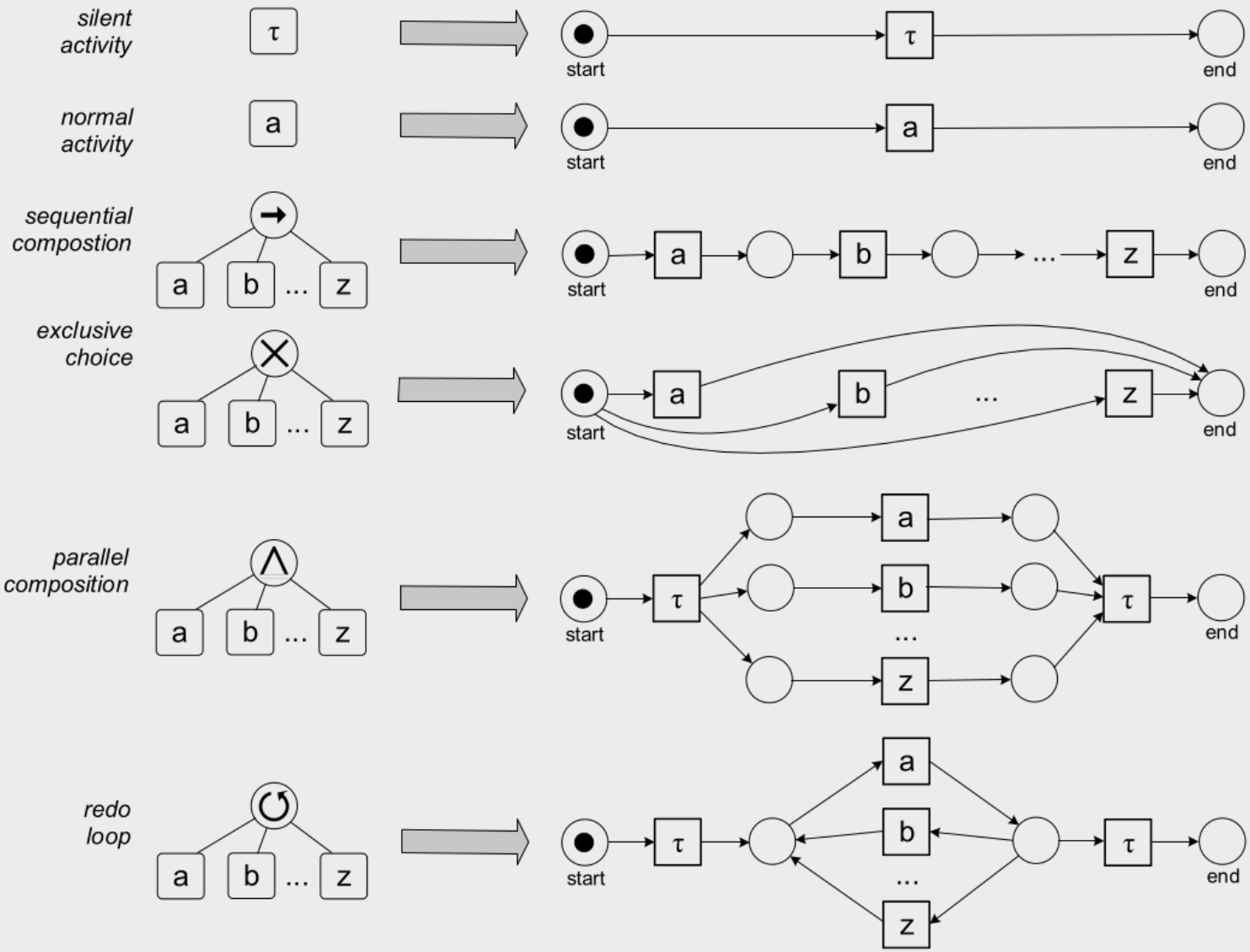
- Wenn $a \in A \cup \{\tau\}$, dann ist $Q = a$ ein Process Tree,
- Wenn $n \geq 1$, $Q_1, Q_2, \ldots, Q_n$ Process Trees und $\oplus \in \{\rightarrow, \times, \wedge\}$, dann ist $Q = \oplus(Q_1, Q_2, \ldots, Q_n)$ ein Process Tree,
- Wenn $n \geq 2$, $Q_1, Q_2, \ldots, Q_n$ Process Trees dann ist $Q = \circlearrowleft(Q_1, Q_2, \ldots, Q_n)$ ein Process Tree,

Process Tree: Beispiel



[Aal16]

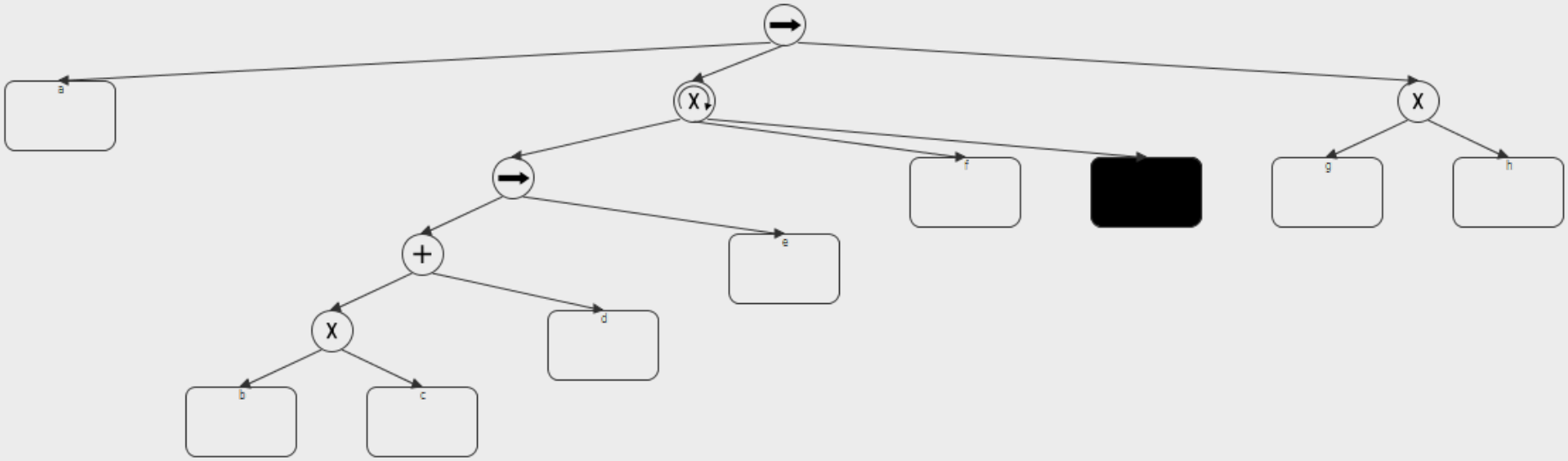
Process Tree: Abbildung auf Workflow-Netze



[Aal16]

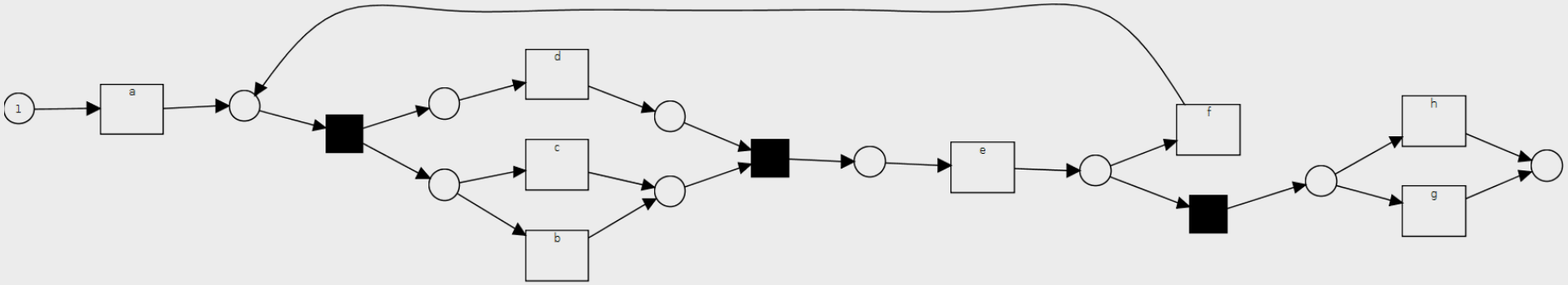
Process Tree in ProM

- Plugin "Start Process Tree Editor Beginner"
- Im Ggs. zum Buch braucht man beim redo Loop hier noch ein drittes Element für die Fortsetzung.



Petri Netz aus Process Tree

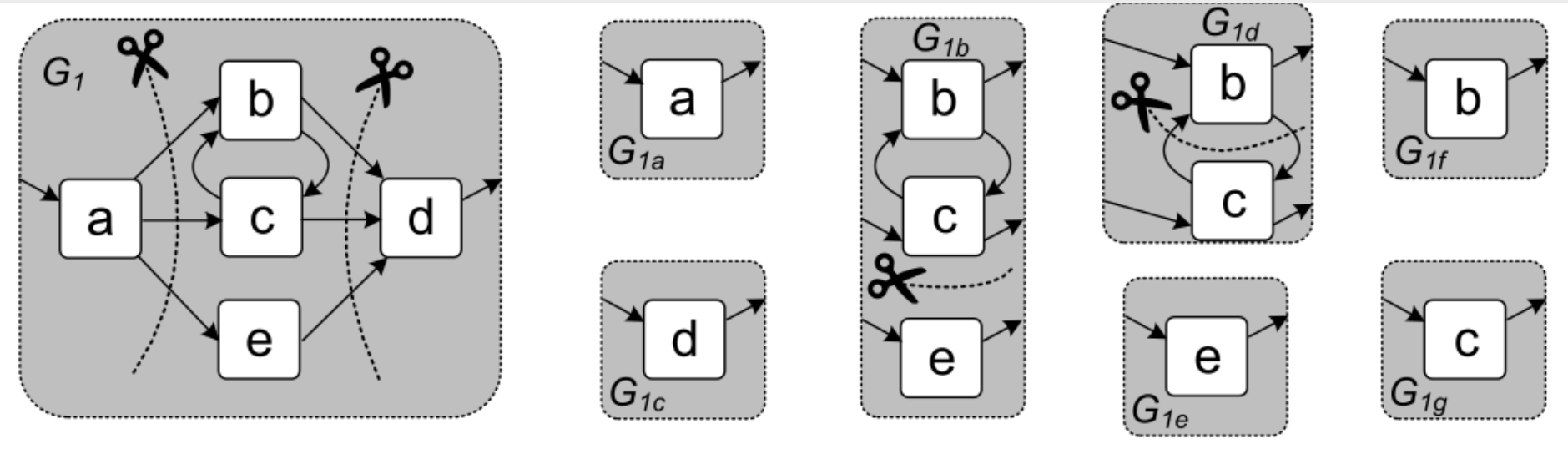
- Plugin "Convert Process Tree to Petri Net"



- Process Trees liegen in ihrer Ausdrucksstärke zwischen Regulären Ausdrücken und Prozess-Kalkülen wie CSP (s. u.).
- Die Semantik von Process Trees kann auch direkt, d. h. ohne den Umweg über Workflow-Netze, definiert werden. [Aal16]

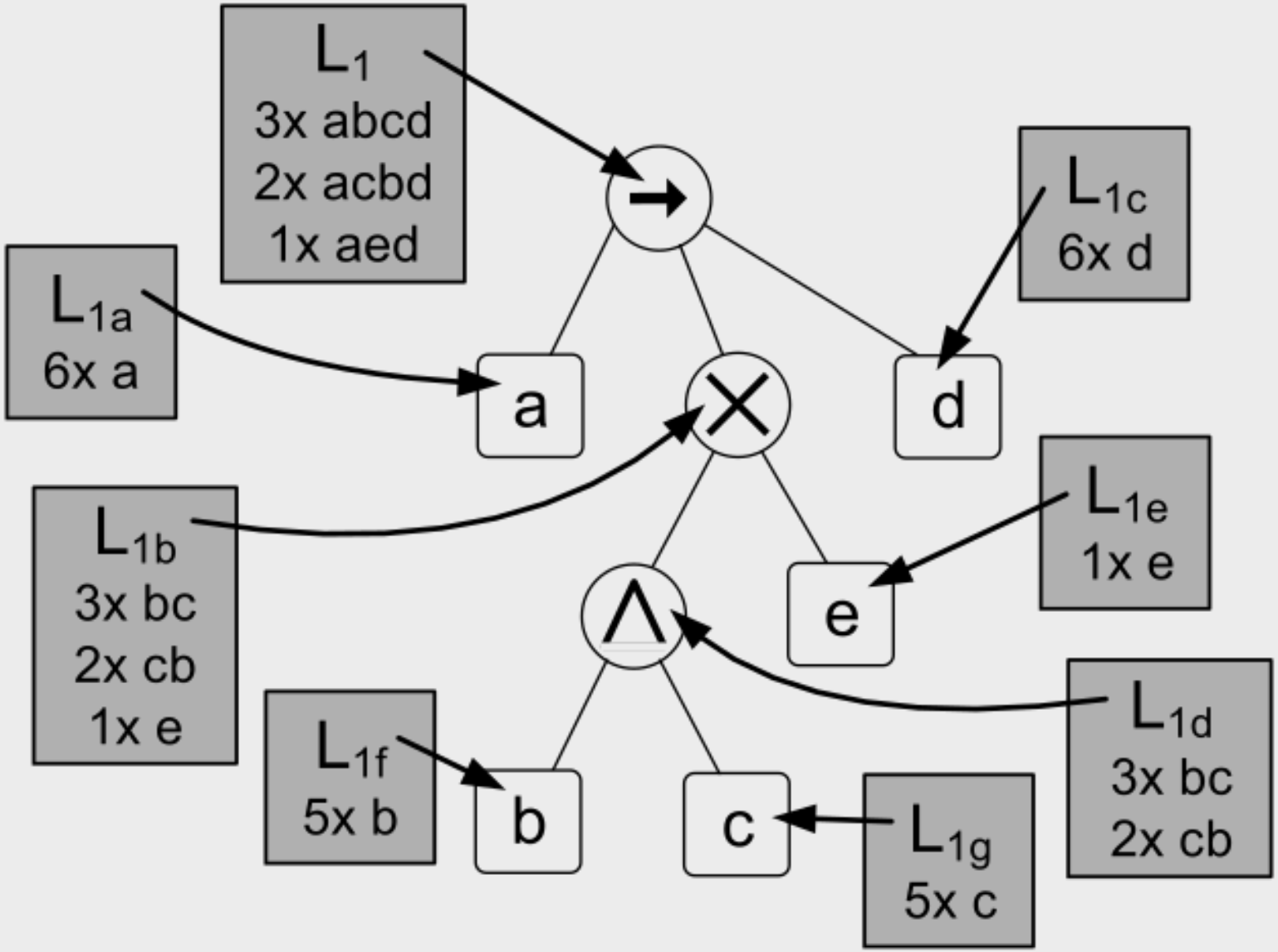
- Wir erklären die Vorgehensweise des Inductive Miners anhand eines bekannten Event Logs.
- $L_1 = [\langle a, b, c, d\rangle^3, \langle a, c, b, d\rangle^2, \langle a, e, d\rangle]$
- (`.../promExperimente/ArtificialLogs/L1.xes`)
- Wir erstellen den "Directly follows graph", der der "folgt direkt" Relation des Alpha-Algorithmus $>_L$ entspricht.
- Wir versuchen nun den "Directly follows graph" mit Schnitten, die den vier Process-Tree Operatoren entsprechen, zu zerteilen.
- Zunächst wird G_1 mit einem Sequence-Cut in G_{1a}, G_{1b}, G_{1c} zerschnitten.
- Jedes Schnipsel hat dann wieder ein eigenes Sublog (s. übernächste Abb.).
- Dann wird G_{1b} mit einem Exclusive-Choice-Cut in G_{1d} und G_{1e} zerschnitten.
- Schließlich wird G_{1d} noch in G_{1f} und G_{1g} zerschnitten.
- Die Sublogs haben jetzt nur noch ein Element.

Directly Follows Graph von L_1



[A

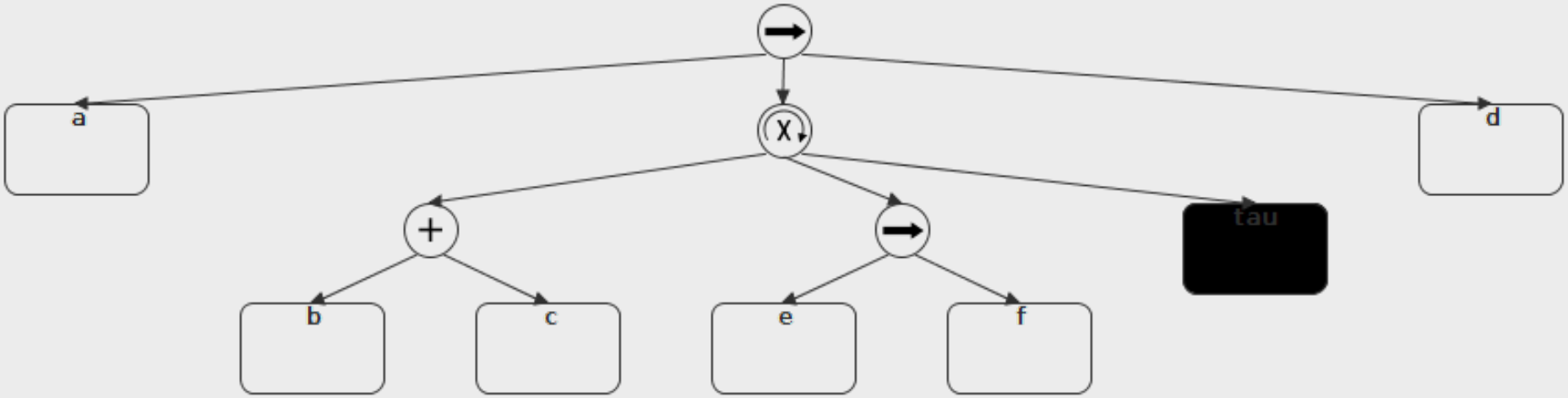
Sublogs von L_1



[Aal16]

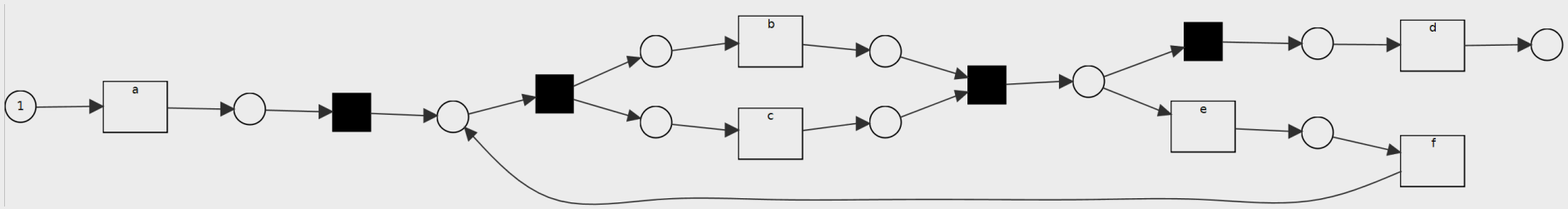
- Die genaue Beschreibung, wie die Cuts erstellt werden, ist in [Aal16] enthalten.
- Wir wollen jedoch trotzdem an einem etwas größeren Beispiel versuchen, einen Process-Tree zu erzeugen.
- Dazu nehmen wir folgenden Log.
- $L_2 = [\langle a, b, c, d\rangle^3, \langle a, c, b, d\rangle^4, \langle a, b, c, e, f, b, c, d\rangle^2, \langle a, c, b, e, f, b, c, d\rangle^2, \langle a, b, c, e, f, c, b, d\rangle, \langle a, c, b, e, f, b, c, e, f, c, b, d\rangle$
- `promExperimente/ArtificialLogs/L2.xes`
- Genaue Lösung im Buch.
- Lösung nächste Folie.

Lösung



- Mit Plugin "Mine Process Tree with Inductive Miner" erstellt.

Trace-äquivalentes Petri-Netz



- Mit Plugin "Convert Process Tree to Petri Net" erstellt.

Process Mining Algorithmen

Fuzzy Miner

- Manchmal ist die schiere Anzahl von Aktivitäten ein Problem.
- Oder es ist kein strukturierter Workflow die Quelle der Logs sondern das freie Verhalten von Nutzern.
- Dann kann es sein, dass der Fuzzy Miner die beste Lösung ist [GA07].
- Unterscheidet nicht zwischen AND und XOR.
- Nutzt eine ähnliche Darstellung wie Causal Nets [AAD11].
- Sehr effizient.
- Wird von Disco und Celonis genutzt.
- Beschreibung siehe [GA07].

- Die Einschätzung der vier o. g. Qualitätskriterien ist manchmal subjektiv.
- Hier wollen wir jetzt behandeln, wie man das Kriterium "Fitness" objektiv bewerten kann.
- Dazu nutzen wir eine Fitness, welche über Alignments definiert ist.
- Dies geschieht für Petrinetze mit stillen Transitionen und ggf. gleichen Labels für unterschiedliche Transitionen.

Definition

Ein Move ist ein Paar $(x, (y, t))$, wobei x ein Element des Logs, y ein Transitionslabel des Modells und t eine Transition des Modells ist.

Falls die Label eines Petrinetzes die Transitionen eindeutig bestimmen, kann man t auch weglassen.

Definition

Ein Move $(x, (y, t))$ ist legal, wenn eine der folgenden Bedingungen gegeben ist:

Synchronous move $x = y$ und y ist das sichtbare Label einer Transition t,

Visible model move $x = \gg$ und y ist das sichtbare Label einer Transition t,

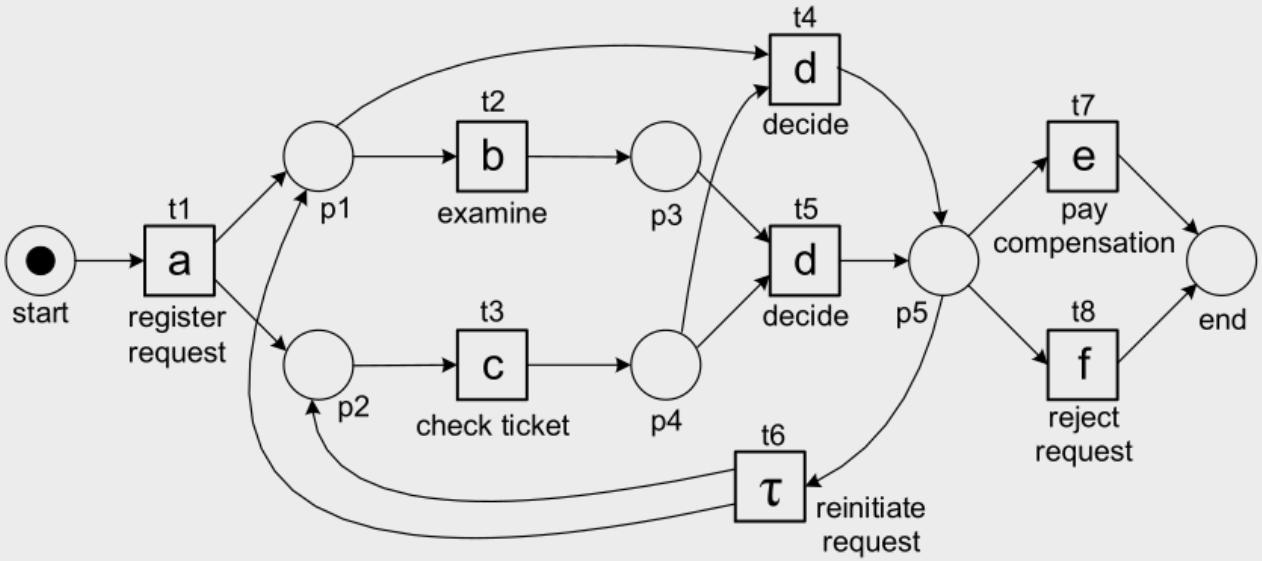
Invisible model move $x = \gg$, $y = \tau$ und Transition t ist unsichtbar,

Log move $x \neq \gg$ und $(y, t) = \gg$.

Definition

Ein Aligment ist eine Sequenz von legalen Moves, so dass – nach Entfernung aller $\gg$ – die obere Reihe einem Trace im Log entspricht, und die untere Reihe eine komplette Feuersequenz des Prozessmodells ist.

Beispiel: Workflow-Netz mit zwei optimalen Alignments



$$\gamma_{5,2a} = \begin{array}{|c|c|c|c|c|} \hline a & b & \gg & d & f \\ \hline a & b & c & d & f \\ t1 & t2 & t3 & t5 & t8 \\ \hline \end{array} \qquad \gamma_{5,2b} = \begin{array}{|c|c|c|c|c|} \hline a & \gg & b & d & f \\ \hline a & c & b & d & f \\ t1 & t3 & t2 & t5 & t8 \\ \hline \end{array}$$

[Aal16]

Definition

Die Kostenfunktion δ weist legalen Moves Kosten zu.

Synchronous move $\delta(x,(y,t)) = 0$ wenn $x = y$,

Invisible model move $\delta(\gg,(\tau,t)) = 0$,

Visible model move $\delta(\gg,(y,t)) = 1$, wenn $y \neq \tau$,

Log move $\delta(x,\gg) = 1$.

- Man kann die Kostenfunktion natürlich auch anders gewichten.
- Ein Alignment ist optimal, wenn es keines mit geringeren Kosten gibt.
- Natürlich kann es mehrere optimale Alignments geben.
- Wir können jedoch davon ausgehen, dass man von diesen immer eines deterministisch aussuchen kann.

Definition

Zu einem Prozessmodell N und einem Trace σ definieren wir die Funktion $\lambda^N_{opt}(\sigma)$, die dem Trace ein optimales Aligment zuordnet.

Definition

Zu einem Prozessmodell N und einem Trace σ definieren wir die Funktion $\lambda^N_{worst}(\sigma)$, die dem Trace ein worst-case Aligment zuordnet. Dieses wird gebildet, indem man zunächst $|\sigma|$ log-only Moves macht und danach model-only Moves auf dem kürzesten Pfad vom Anfangszustand zum Endzustand des Modells durchführt.

Definition

Die Fitness eines Trace σ wird definiert als:

$$fitness(\sigma, N) = 1 - \frac{\delta(\lambda^N_{opt}(\sigma))}{\delta(\lambda^N_{worst}(\sigma))}$$

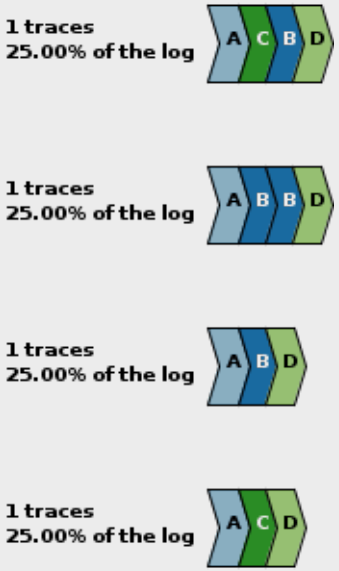
Definition

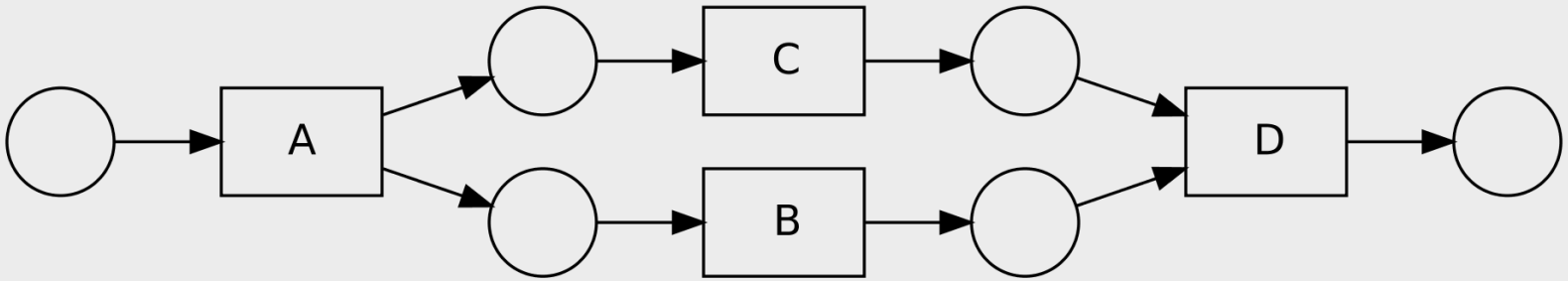
Die Fitness eines Logs L wird definiert als:

$$fitness(L, N) = 1 - \frac{\sum_{\sigma \in L} L(\sigma) \times \delta(\lambda^N_{opt}(\sigma))}{\sum_{\sigma \in L} L(\sigma) \times \delta(\lambda^N_{worst}(\sigma))}$$

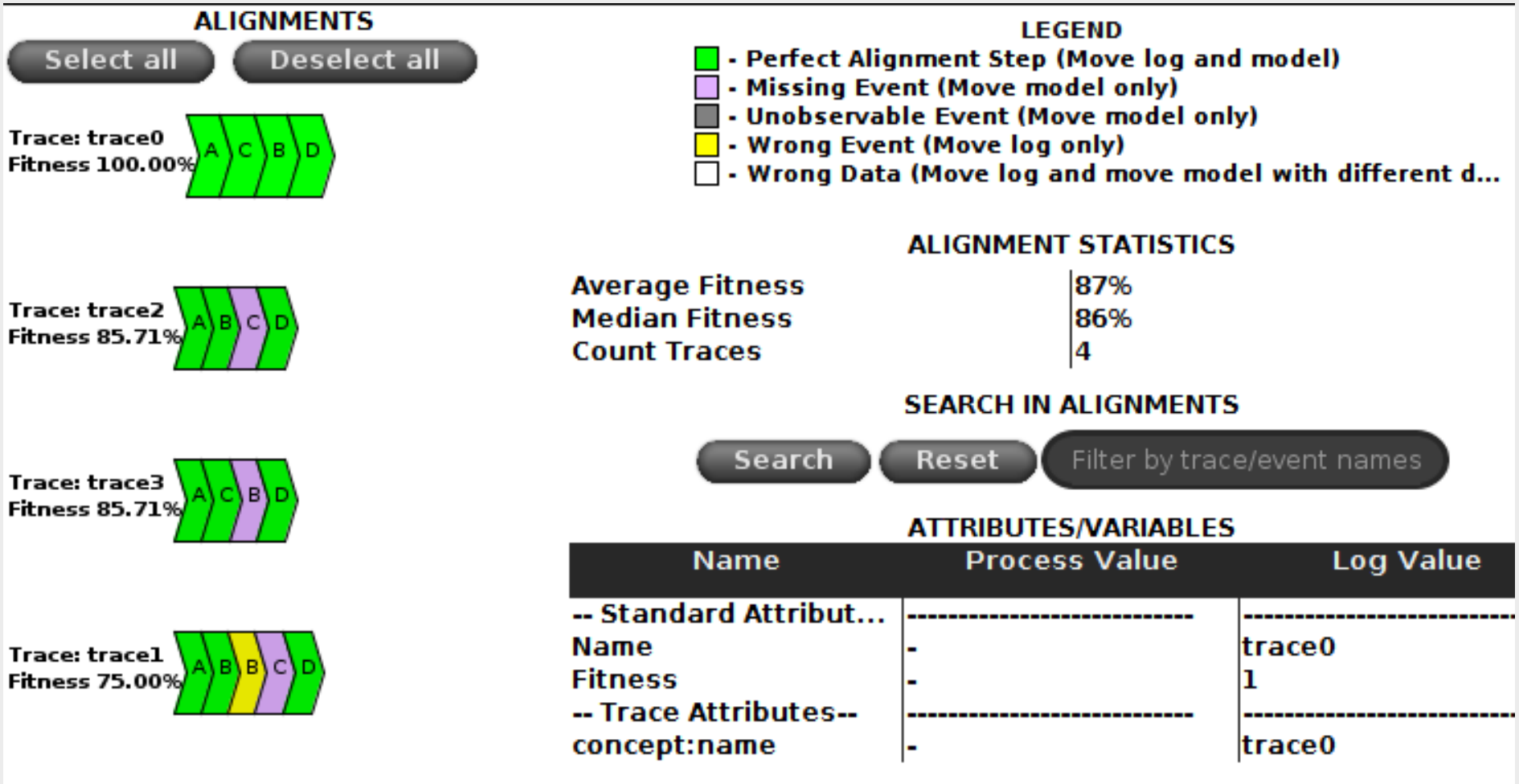
- ProM Action: Conformance Checking of DPN
- berechnet die Fitness auf Basis von Alignments
- Input: Log, Petrinetz

Berechnen Sie die Fitness!





Antwort (Average Fitness)



Case Studies

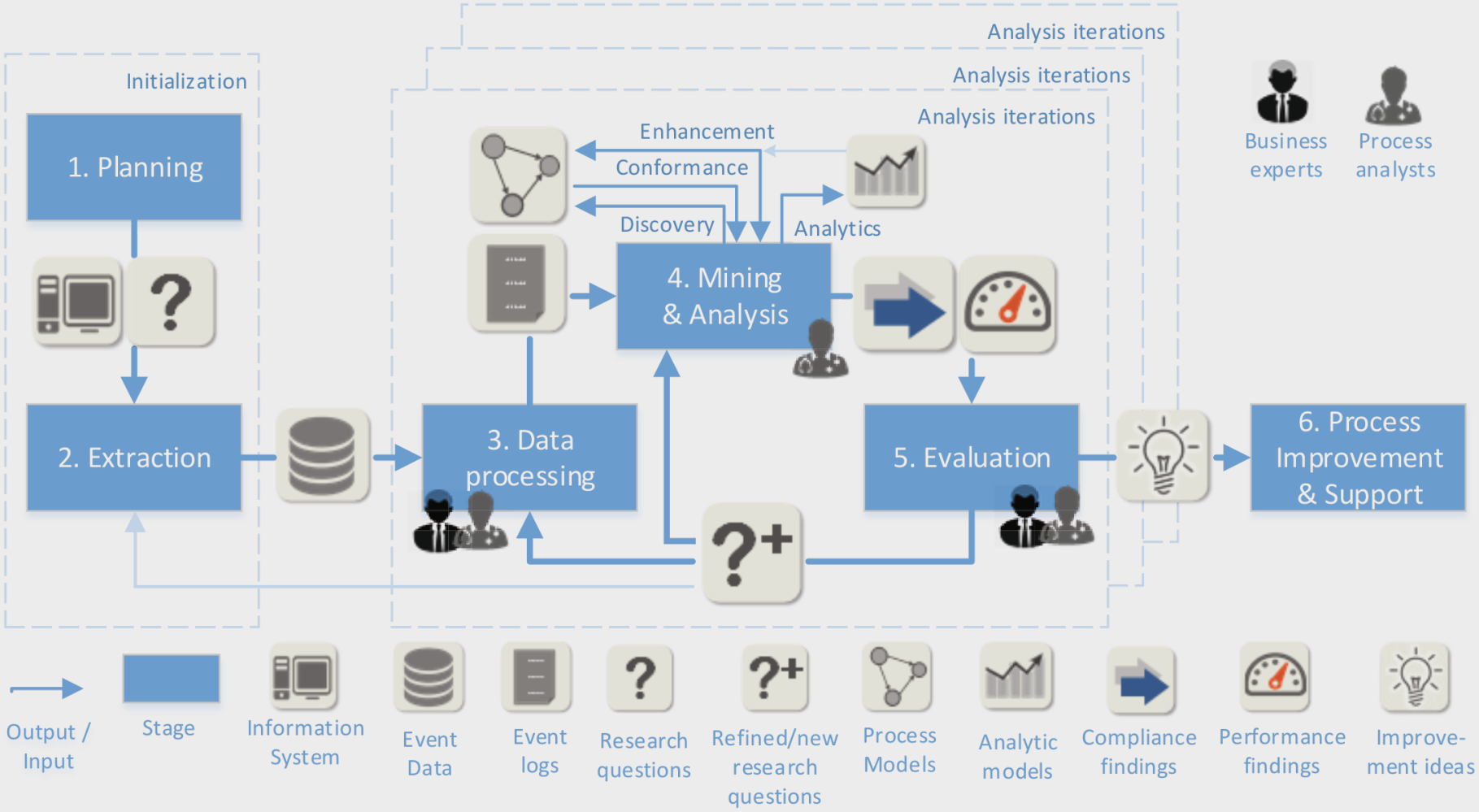
BPI Challenge

- http://www.win.tue.nl/bpi/doku.php?id=2011:challenge
- http://www.win.tue.nl/bpi/doku.php?id=2012:challenge
- http://www.win.tue.nl/bpi/doku.php?id=2013:challenge
- http://www.win.tue.nl/bpi/doku.php?id=2014:challenge
- http://www.win.tue.nl/bpi/doku.php?id=2015:challenge
- http://www.win.tue.nl/bpi/doku.php?id=2016:challenge
- https://www.win.tue.nl/bpi/doku.php?id=2017:challenge
- https://www.win.tue.nl/bpi/doku.php?id=2018:challenge
- https://icpmconference.org/2019/icpm-2019/contests-challenges/bpi-challenge-2019/
- https://icpmconference.org/2020/bpi-challenge/

IEEE Case Studies

http://www.win.tue.nl/ieeetfpm/doku.php?id=shared:process_mining_case_studies

Vorgehensmodell



[ELLA15]

Process Mining Tools – open source

ProM Forschungsplattform der TU Eindhoven. Open source Software.
`http://www.promtools.org`

bupaR Integrierte Suite von R-Paketen für Daten von Geschäftsprozessen
`https://www.bupar.net/`

Process Mining Tools – kommerziell

Disco Kommerzielles Process Mining Tool mit Heuristic Miner. Flexibler Import von Tabellenkalkulation. Umformung in XES. `https://fluxicon.com/academic`

Celonis Kommerzielles Process Mining Tool – auch SaaS. `https://my.celonis.de/academic_signup`

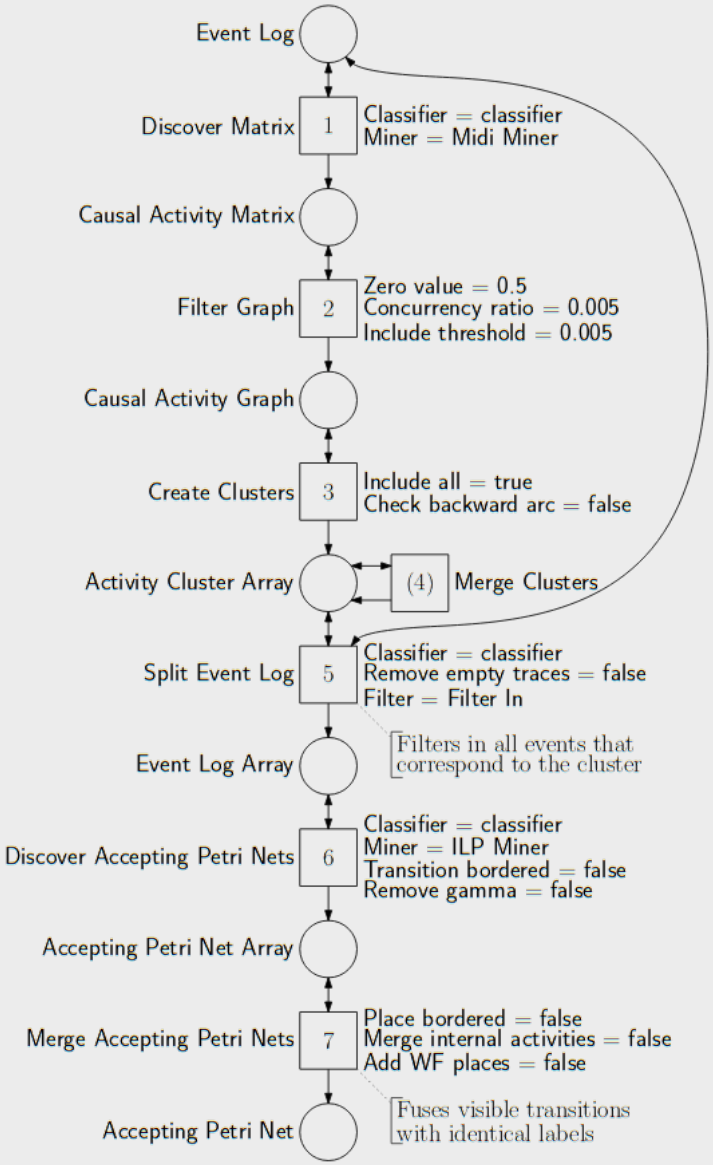
Werkzeuge

- PostgreSQL, pgfutter
- awk, sed, Excel/Calc
- ProM, ProM nightly builds
- WoPeD, YAWL

Dokumentation

- Die durchgeführten Schritte und die Zwischenergebnisse müssen dokumentiert werden.
- Die nächste Folie zeigt, wie dies aussehen kann.
- Natürlich geht das auch als Tabelle.

Process Mining Schritte als Petrinetz



[VA14]

Literatur I

Aalst, Wil van d. ; Adriansyah, Arya ; Dongen, Boudewijn v.:
Causal Nets: A Modeling Language Tailored towards Process Discovery.
In: *CONCUR 2011 – Concurrency Theory*, Springer, Berlin, Heidelberg, September 2011, 28–42

Aalst, Wil M. P. van d.:
Process Mining: Data Science in Action.
2nd ed. 2016.
New York, NY : Springer, 2016. –
ISBN 978–3–662–49850–7

Literatur II

Eck, Maikel L. ; Lu, Xixi ; Leemans, Sander J. J. ; Aalst, Wil M. P. d.:
PM2: A Process Mining Project Methodology.
In: Zdravkovic, Jelena (Hrsg.) ; Kirikova, Marite (Hrsg.) ; Johannesson, Paul (Hrsg.): *Advanced Information Systems Engineering*, Springer International Publishing, 2015 (Lecture Notes in Computer Science). –
ISBN 978–3–319–19069–3, S. 297–313

Günther, Christian W. ; Aalst, Wil M. P. van d.:
Fuzzy Mining – Adaptive Process Simplification Based on Multi-perspective Metrics.
In: *Business Process Management*, Springer, Berlin, Heidelberg, September 2007, 328–343

Literatur III

Verbeek, H. M. W. ; Aalst, Wil M. P. van d.:
Decomposed Process Mining: The ILP Case.
In: *Business Process Management Workshops*, Springer, Cham, September 2014, 264–276

Weijters, A. J. M. M. ; Ribeiro, J. T. S.:
Flexible Heuristics Miner (FHM).
In: *2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*, 2011, S. 310–317

Hinweise zur Nutzung dieses Foliensatzes

Dieser Foliensatz wird ausschließlich den Teilnehmern dieser Lehrveranstaltung zur Verfügung gestellt. Er dient als Hilfe für die Vor- und Nachbereitung der Inhalte der Lehrveranstaltung und zur Prüfungsvorbereitung. Für diese Zwecke darf eine lokale Kopie erzeugt werden. Dieser Foliensatz darf nicht an Dritte weitergegeben werden.

© Andreas V. Hense 2025